

AIFS



Australian Government
Bureau of Meteorology

Hydrology Raw Data Filtering Program Specification

May 2009

Version 2.1

**Hydrology Raw Data Filtering Program Specification,
version 2.1**

Date of Issue: May 2009

Copyright: © 2009 by Commonwealth Bureau of Meteorology

All rights reserved. No part of this document may be reproduced by any means or in any form without the express permission of the Commonwealth Bureau of Meteorology.

This documentation was prepared for Commonwealth Bureau of Meteorology by:

Wordware Pty Ltd
1/321 Whitehorse Road
Nunawading VIC 3131



Table of Contents

Table of Contents.....	3
Background.....	5
Analysis.....	6
Roll-over Data.....	6
Noise Data.....	6
System Design.....	7
Algorithm.....	7
<i>Establish Start Continuity.....</i>	<i>8</i>
<i>Establish Continuity.....</i>	<i>12</i>
Object Design.....	16
hyFilter.....	16
Implementation.....	18
Testing.....	19
Creating Data File.....	19
Executing 357-Filter program.....	19

List of Figures and Tables

Figure 1: Data Flow Diagram of raw data filtering process and the related components.....	5
--	----------

Figure 2: Top level flow chart of the 357-filter process.....	7
--	----------

Figure 3: Different bad data patterns may occur in the StartContinuity window.....	9
---	----------

Figure 4: Establishing Start Continuity.....	10
Figure 5: Establishing Start Continuity with bad data.....	11
Figure 6: Flowchart of Start Continuity algorithm.....	12
Figure 7: Establishing Continuity.....	13
Figure 8: A new data series established after 4 consecutive 3-5-7 tests fail.....	14
Figure 9: Flowchart of Establish Continuity algorithm.....	15
Table 1: hyFilter attributes.....	16
Table 2: hyFilter methods.....	16
Table 3: Components of the 3-5-7 filter module	18



Background

The rainfall and water height data (event time-series data) collected from the observation stations via the ERTS (Event-reporting Radio Telemetry System) protocol may contain noise, missing data (during transmission) or roll-over data (when accumulator value reaches the limitation of the observation hardware device). These data can confuse the forecasting systems and produce incorrect forecasting results. Therefore, the raw observation filtering process is used to improve the quality of observation data from ERTS.

The filtering process applies the Filter 3-5-7-algorithm to detect and remove bad data from the ERTS data based on the continuity pattern of the data set. This Filter 3-5-7 algorithm is reproduced from the existing C program that is used in ALERT Data Collection named **contest357**.

The filtering program retrieves raw data (RainAccum, WaterLevel) from the AXF file that does the filtering process and stores the filtered data back to the AXF file in the filtered section (see Figure 1).

Figure 1: Data Flow Diagram of raw data filtering process and the related components

Analysis

The ERTS data is generated in a continuous form. The continuous form of the ERTS data is often broken due to the presence of noise, loss of data and interruption of data transmission. The filter process has to identify and remove noise data, adjust rollover data and establish the continuity of data.

Therefore, the filter process should not simply compare the difference between the data, but should also test for the data patterns when making a decision about whether or not to accept the suspect data.

For example, a large gap of the data may not always indicate an error, but could be an interruption of the transmission over a long period of time. In such a case, the continuity of the data is broken and the filter process has to re-establish a new set of time-series data.

Any four or more consecutive data forms a continuous set of data.

Roll-over Data

The ERTS observation devices can only count from 0 to 2047. Therefore, the accumulator value is rolled over to 0 after it reached the cycle size of 2047. In some cases, the data is rolled over before it has reached the cycle size because the observation devices have been deliberately reset.

Noise Data

Noise data may occur due to interference during transmission or due to faulty observation equipment. Noise data can be a single spike or a group of spike data (refer to *A continuity filter for ERTS data*, Soori Sooriyakumaran, BoM, January 1999).

System Design

The 3-5-7 filter utility program consists of a filter driver program and a filter library.

- **Filter driver program**

The filter driver program provides a command line interface that gets the input arguments from the user. It then loads the data from the AXF file and invokes the filter library to filter the data and write the filtered data back to the AXF file.

- **Filter library**

The filter library accepts the data set from the filter driver program, then goes along the data set and marks all bad data before returning the data set back to the caller. The filter driver program removes the bad data.

Algorithm

The filter algorithm has two components: Establish Start Continuity and Establish Continuity.

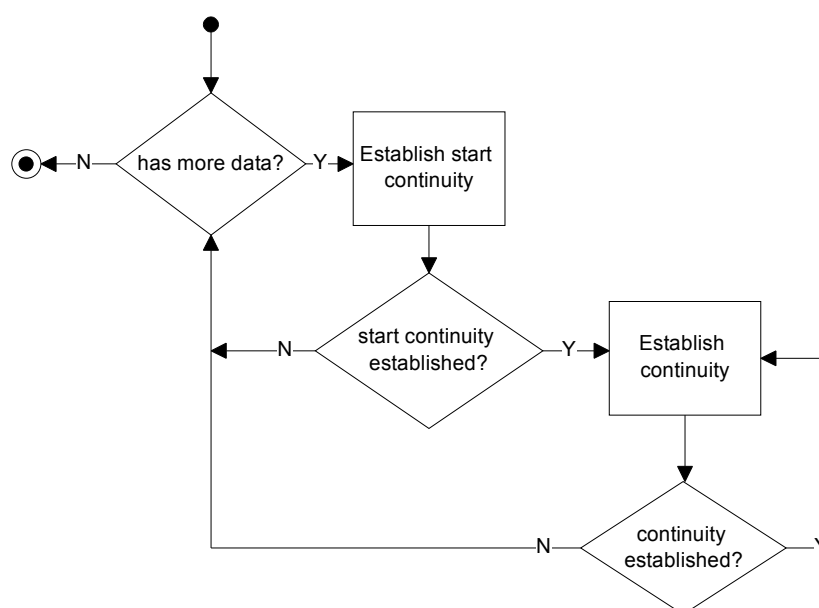
- **Establish Start Continuity component**

The Establish Start Continuity component is involved at the beginning of a time series (when the program is started or the continuity of a time-series data is broken).

- **Establish Continuity component**

The Establish Continuity component is involved right after the start continuity of a new series data has been established (see Figure 2).

Figure 2: Top level flow chart of the 357-filter process



Both Establish Start Continuity and Establish Continuity components use different combinations of 3-5-7 tests to determine which data should be removed from the data set. A data item passes the 3-5-7 test if its value is:

- ≤ 3 when compared with the next data
or
- ≤ 5 when compared with the next-next data
or
- ≤ 7 when compared with the next-next-next data.

There are three types of data used in the Filter 3-5-7 algorithm:

- **Good data**

This is the data that passes the 357 test. This data is kept after the filter process.

- **Bad data**

This is the data that fails the 357 test and other subsequent tests. This data is removed after the filter process.

- **Suspect data**

This is the data that fails the 357 test and is waiting for other subsequent tests. This data becomes good data if it passes subsequence tests or becomes bad data if it fails the subsequent tests.

If the data is marked as **Bad**, then it is ignored and will not be involved in any further 3-5-7 test or subsequent tests.

Data is stored in ascending time order, with the latest data at the end of the list. The Filter 3-5-7 algorithm starts from the end and finishes at the beginning of the data list.

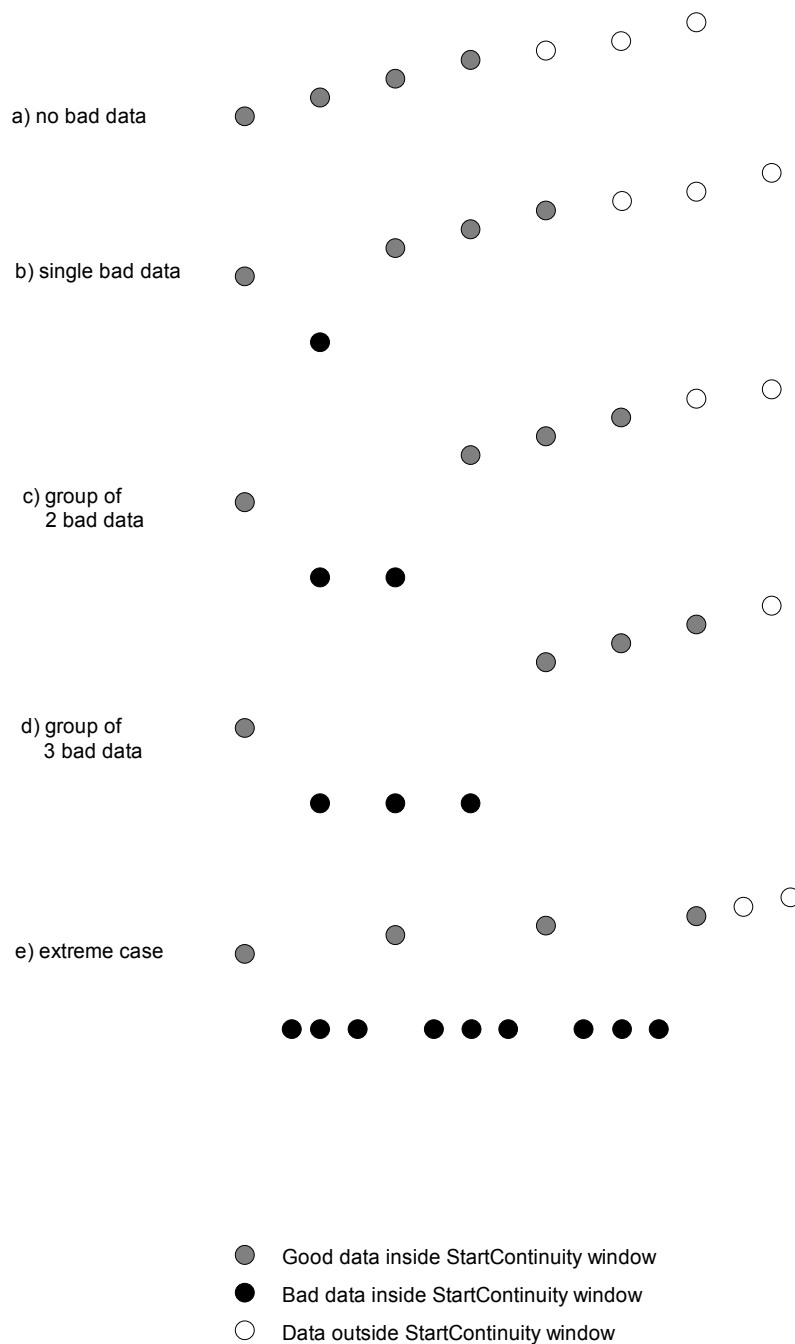
The 3-5-7 test involves two data on the list for the test:

- The data item being tested is called **Current Data**.
- The adjacent data item toward the end of the list is called **Next Data**.
The adjacent data item toward the beginning of the list is called **Previous Data**.

Establish Start Continuity

The Start Continuity (beginning of a data series) is established when there are at least four good data items in the data set (see Figure 3a). No more than three bad data in a group can occur in the *Start Continuity* window (see Figure 3b-e).

Figure 3: Different bad data patterns may occur in the StartContinuity window



The process is as follows:

1. The Establish Start Continuity starts at the end of the list.
2. The current data is tested with the start data. If the difference between current data and start data is not greater than **3** then the current data is marked as **Good**.
3. The value of current data is then obtained from the previous position and is tested with the start data. If the difference between current data and start data is not greater than **5**, then the current data is marked as **Good**.

4. The value of current data is then obtained from the previous position and is tested with the start data. If the difference between current data and start data is not greater than 7, then the current data is marked as **Good**.
5. At this stage, the Start Continuity is established because there are four good consecutive data on the list. The Establish Continuity continues the test from the current position until the continuity is broken again or no more data on the list (see Figure 4).

Figure 4: Establishing Start Continuity

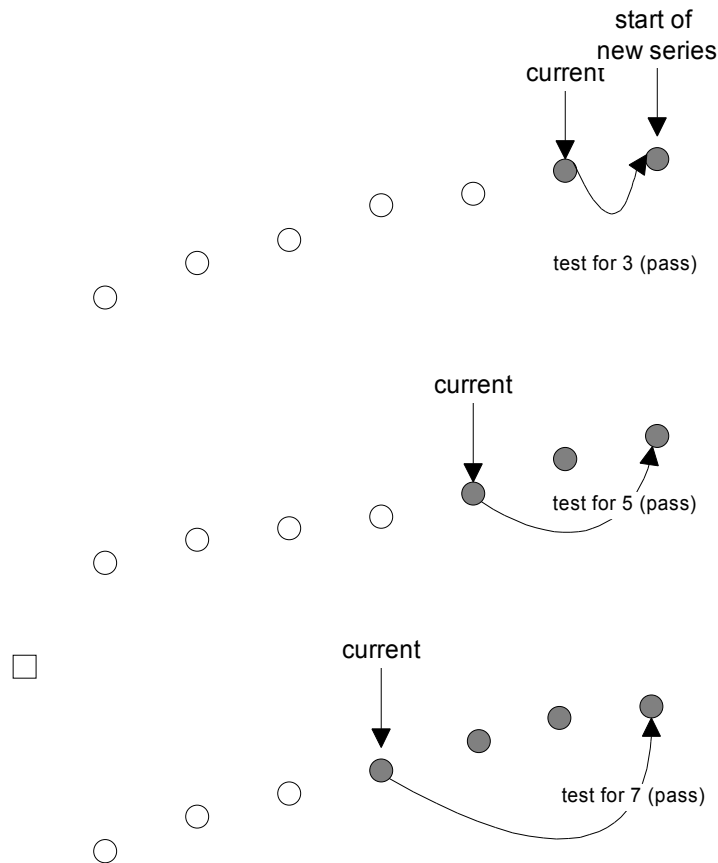




Figure 5: Establishing Start Continuity with bad data

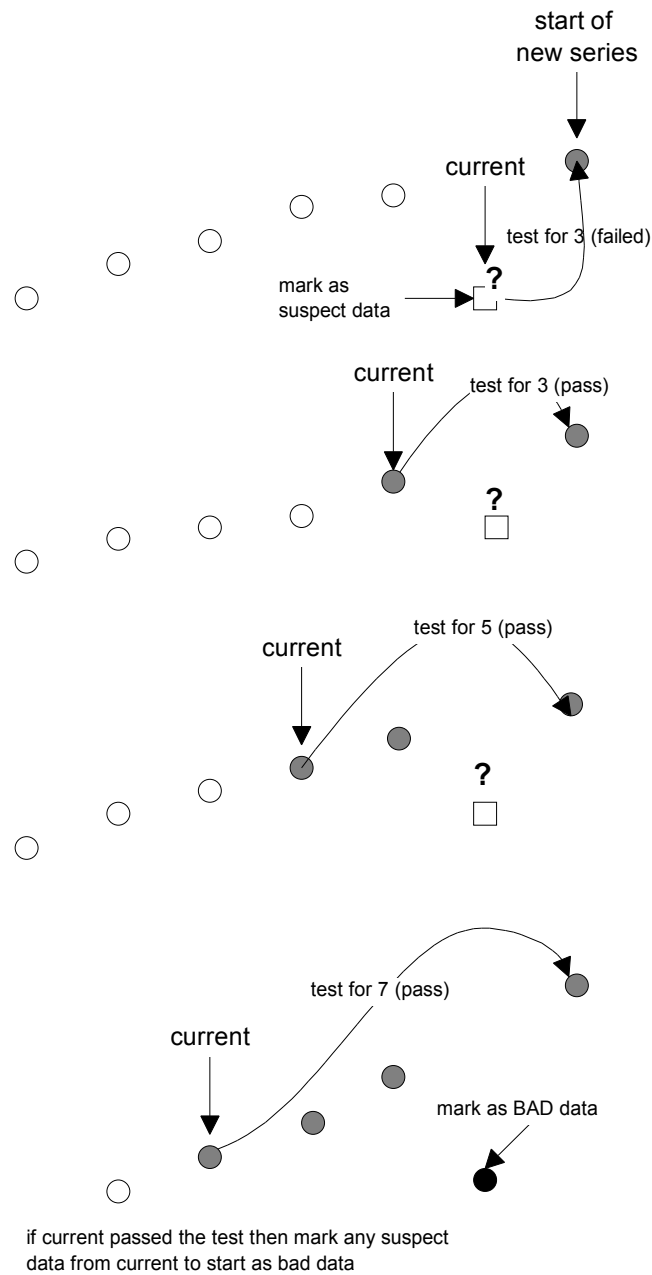
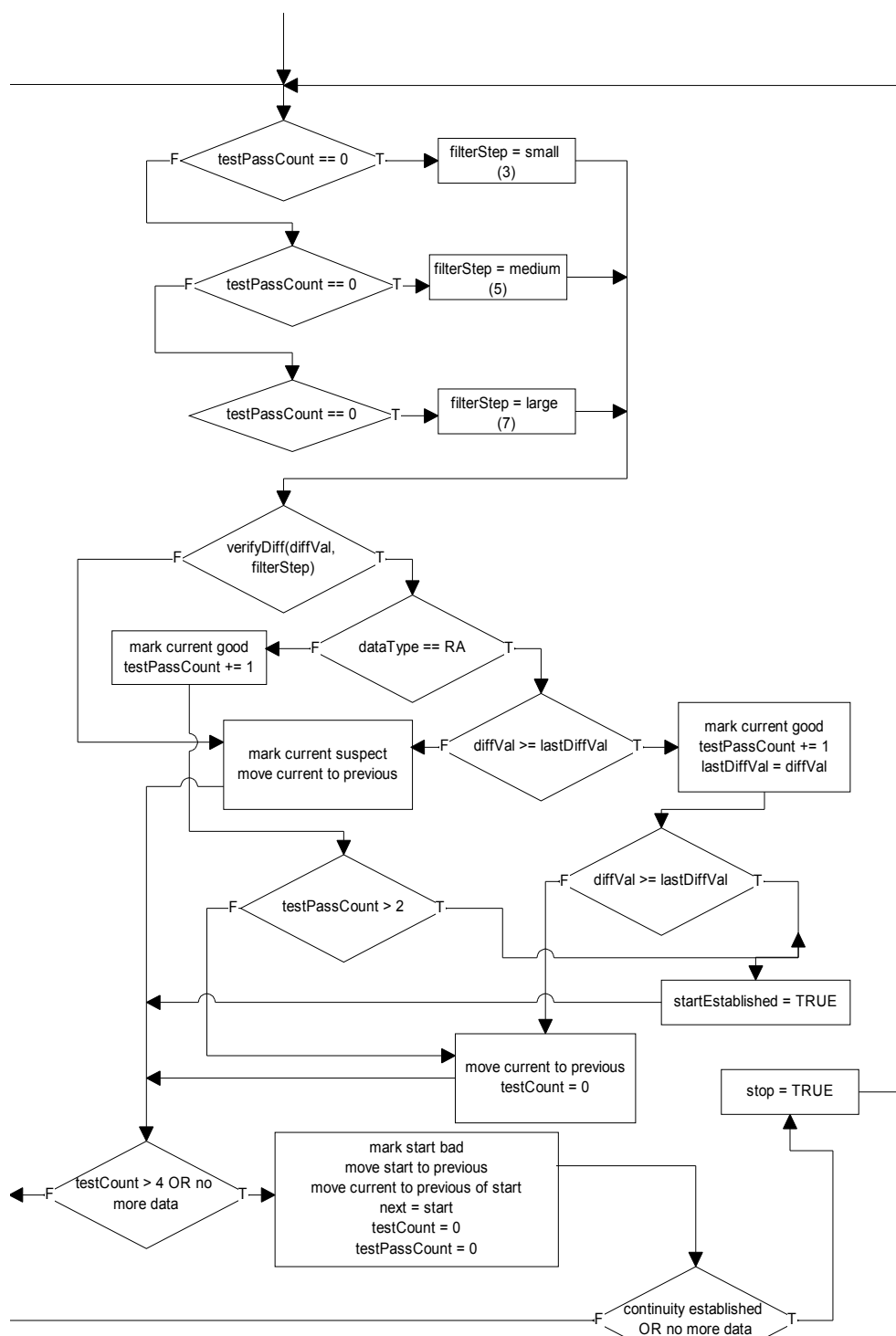


Figure 6: Flowchart of Start Continuity algorithm



Establish Continuity

The Establish Continuity process is involved right after the start continuity has been established.

- If the current data fails the 3-5-7 test with the next data in small step (3), then it is tested with the next-next and next-next-next data for medium step (5) and large step (7).
- If all the tests are failed, the current data is marked as **Suspect** and the current data for the next 3-5-7 test is the previous of the suspect data.
- If the current data passes the test, then all suspect data from the current to “last good” data is marked as **Bad** (see Figure 7).
- The continuity is broken if four consecutive data fail the 3-5-7 test (see Figure 8). The data list is handed over to the Establish Start Continuity process for re-establishing a new time series (start from the first failed data).

Figure 7: Establishing Continuity

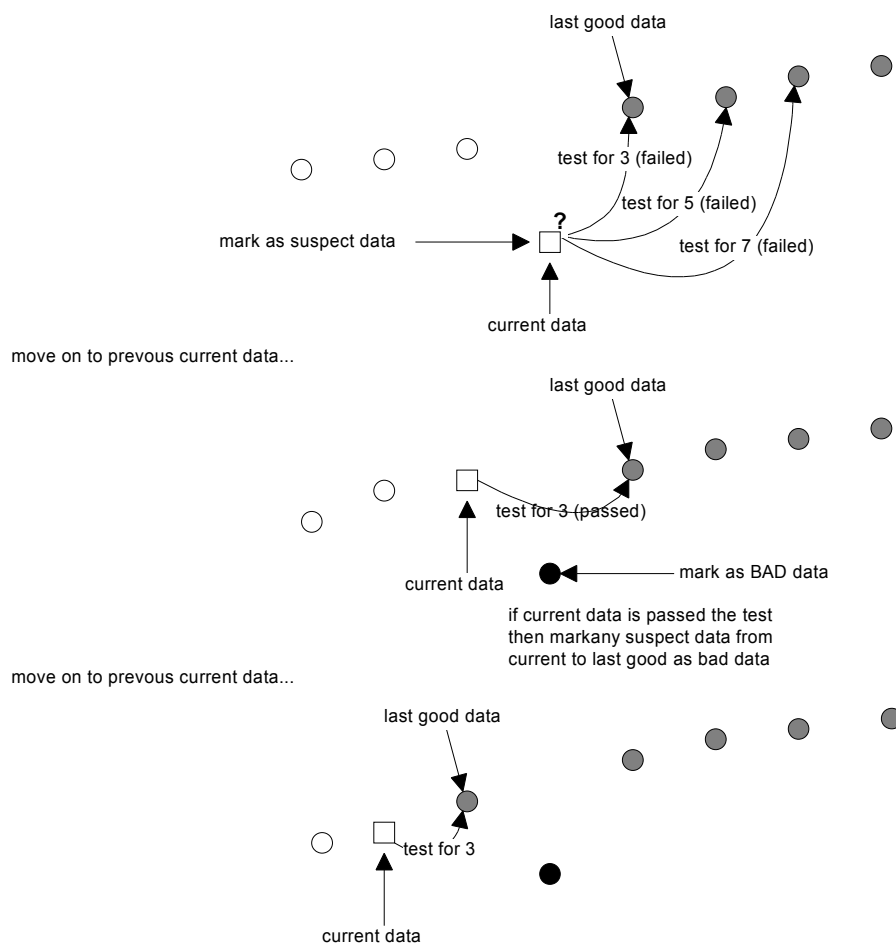


Figure 8: A new data series established after 4 consecutive 3-5-7 tests fail

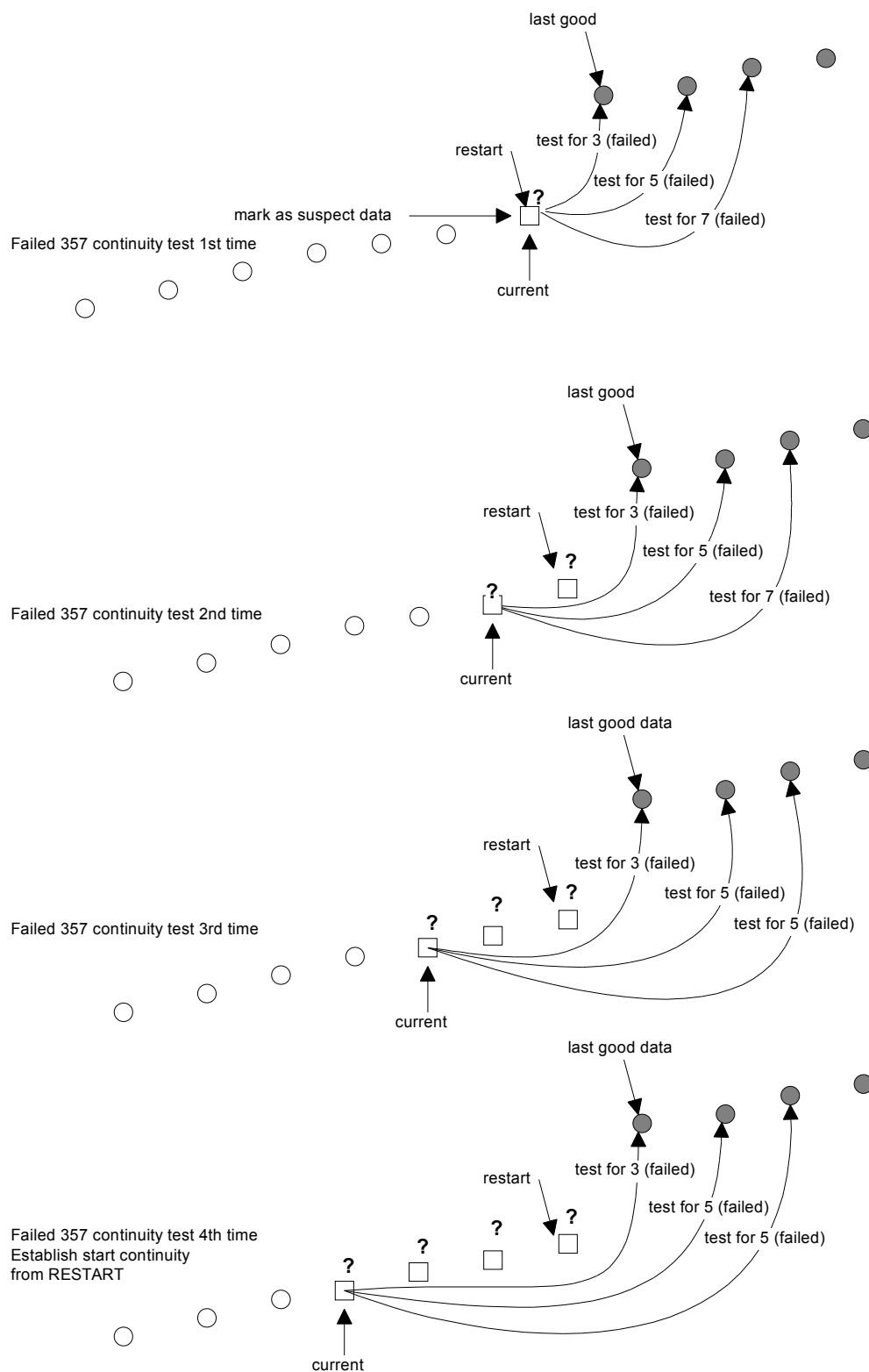
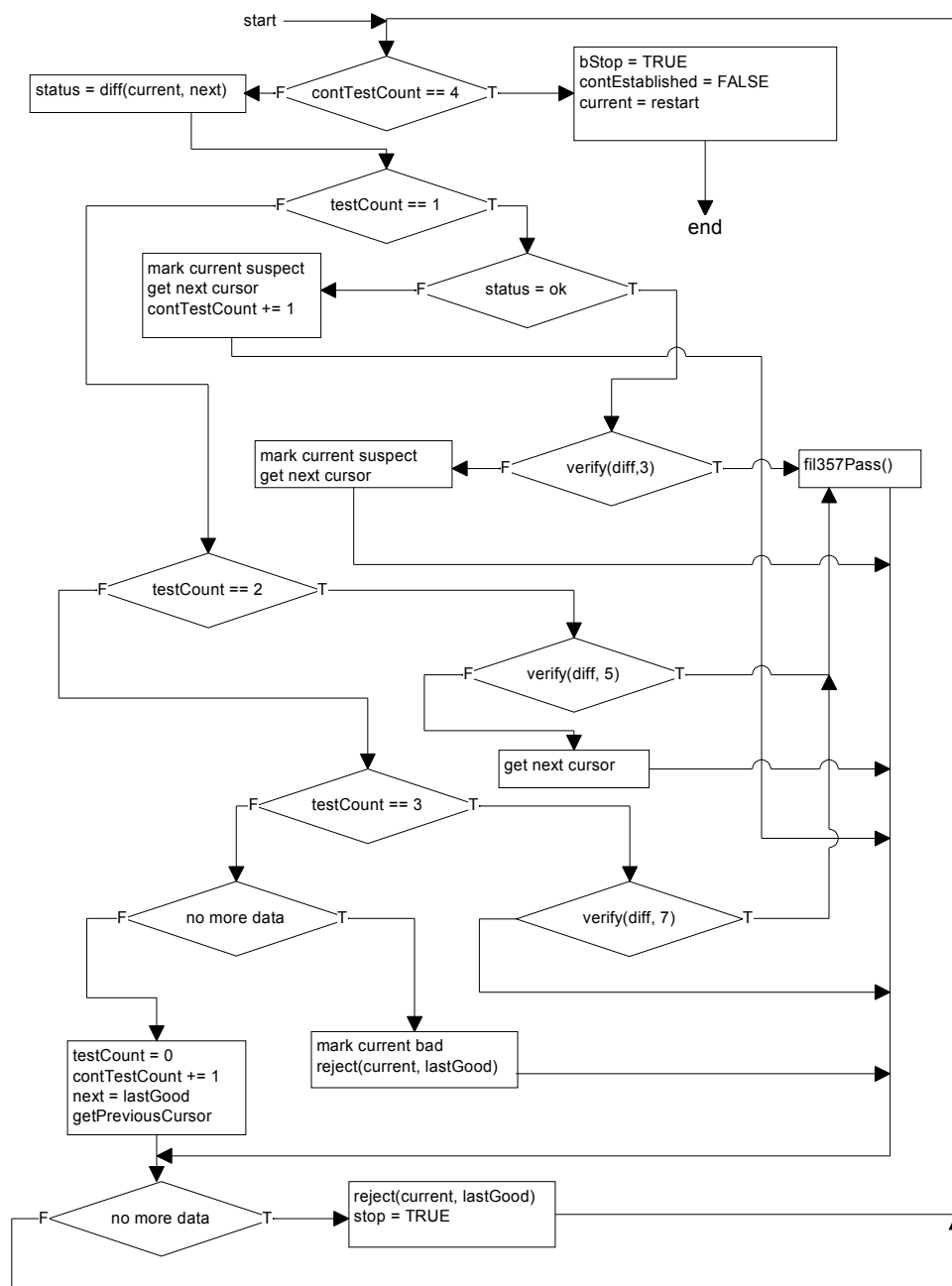


Figure 9: Flowchart of Establish Continuity algorithm



Object Design

hyFilter

Base class None

Table 1: hyFilter attributes

Name	Type	Description
aGenericRawList	HyList<hyGenericRaw>	A list of GenericRaw
aRainAccumList	HyList<hyRainAccum>	A list of RainAccum
aWaterLevelList	HyList<hyWaterLevel>	A list of WaterLevel
smallStep	int	Small step filter (3)
mediumStep	int	Medium step filter (5)
largeStep	Int	Large step filter (7)
cycleSize	long	Limit value for counter (2048)

Table 2: hyFilter methods

Name	Type	Description
Filter357()		Constructor
establishStartContinuity()	int	Establish Start Continuity
establishContinuity()	int	Establish Continuity
filterOutOfSyncDate()	int	Remove data that has out of sync. date
fil357Pass()	int	Mark current data as Good and mark any suspect data from current to last good as Bad
verifyDiff()	int	Return TRUE if different value is not greater than the given step test (3-5-7)
diff()	int	Calculate different value of the two data according to data type (RA or WL)
adjustRolloverData()	int	Adjust the roll-over data



Name	Type	Description
deleteBadData()	int	Go through the list and delete any data has status as Bad
getPreviousCursor()	Int	Get previous position of the given position (skip bad data)
getNextCursor()	Int	Get next position of the given position (skip bad data)
isAtFirst()	int	Return TRUE if given position is at the beginning of the list
isAtLast()	int	Return TRUE if the given position is at the end of the list
mark()	int	Mark the given data with (Good , Suspect or Bad) status
reject()	int	Mark any suspect data as bad data from the given start and end positions
convertToGenericRawList()	int	Convert RA or WL list into GenericRaw list
convertFromGenericRawList()	int	Convert GenericRawList into RA or WL list

Implementation

This 3-5-7 filter module is implemented in C++ and compiled by the IBM C++ compiler (xLC). The sources, libraries and executable files are located on servb machine.

servb: /home/ho/hydro/alpha/hydro

```

    |_include
    |_lib
    |_bin
    |_libsrc
    |      |_hyFilter
    |_src
          |_hyFilter

```

Table 3: Components of the 3-5-7 filter module

Component	File Name	Location
Driver program source files	hyFilterProc.h	Include
	hyFilterProc.cpp	src/hyFilter
Filter library source files	hyFilter.h	include
	hyFilter.cpp	libsrc/hyFilter
Filter library	libhyfilter.a	lib
Filter exe file	hyFilter	bin



Testing

Creating Data File

1. Run the extract util programs `hdExtractRA` or `hdExtractWL` to extract RainAccum or WaterLevel data from database to AXF file.

For example,

```
../hyrtdb/src/hdExtractRA/hdExtractRA -f data.axf -n 587037 -s 19990215000000 -d R -u
```

Executing 357-Filter program

The command line of `hyFilter` is:

`hyFilter`

```
-f <data file name>
-s [start time (yyyymmddhhmmss)]
-e [end time (yyyymmddhhmmss)]
-u (use UTC time)
-r [hdro axf metadata state] (RAW|TRN|DRV - default RAW)
-d [sensor data type -RA|WL]
-q [filter step size] (default 1)
-v (0|1|2|3) Verbose mode
```